

# IT Workshop

Course Code: CS2204

Lecture 3 and  
Lab

Dr. Rahul Mishra

IIT Patna



\* `grep`: Searching for a pattern

⇒ `grep [options] pattern filename(s)`

① `$ grep "Sales" emp.lst`

② `$ grep president emp.lst` # no quoting necessary here

③ `$ grep "director" emp1.lst emp2.lst`

④ `$ grep 'jai sharma' emp.lst`

\* `grep options` :-

\* Ignoring Case (-i)

⑤ `grep -i 'agrawal' emp.lst`



\* Deleting Lines (-v) → grep can play an inverse role too.

[2]

⑥ `grep -v 'director' emp.lst > otherlist`

⑦ `wc -l otherlist`

⑧ \* Displaying Line Number (-n)

The -n (number) option displays the line numbers containing the pattern, along with the lines:

\$ `grep -n 'marketing' emp.lst`

⑨ \* Counting Lines Containing Pattern (-c)

The -c (count) option counting the number of lines containing the pattern (which is not the same as number of occurrences).

\$ `grep -c 'director' emp.lst`

⑩ `grep -c director emp*.lst`

⑪ Displaying Filenames (-l): The -l (list) option displays only the names of files containing the pattern:

`$ grep -l 'manager' *.lst`

⑫ Matching Multiple Patterns (-e)

With the -e option, you can match the three aganwals by using grep like this:

`$ grep -e "Aganwal" -e "agganwal" -e "aganwal" emp.lst`

⑬ Taking Patterns from a File (-f) You can place all the three patterns in a separate file, one pattern per line. grep uses the -f option to take patterns from a file.

`$ grep -f pattern.lst emp.lst`

## The character class :-

[4]

A regular expression lets you specify a group of characters enclosed within a pair of rectangular brackets, **[ ]**

> **[a|o]** # matches either on **(a)** or on **(o)**

> The metacharacters **[** and **]** can now be used to match **agawal** and **agrawal**.

**[aA]g[an][an]wal**

> The model **[an][an]** matches any of the four patterns:

aa on na nn

> **\$ grep "[aA]g[an][an]wal" emp.lst**



## Negating a class (^) :-

> Regular expression use the ^ (caret) to negate the character class, while the shell use the !

>  $[^a-zA-Z]$  matches a single nonalphabetic character string.

>  $[^p]$  represents any character other than p.

\* Regular expressions take care of some common query and substitution requirement.

\* Example  $\rightarrow$  \* You may want the system to present a list of similar names so you can select exactly the one you need.

\* You may want to replace multiple spaces with a single space, or display lines that begin with a #.

\* You may even be looking for a string at a specific column position on line

- \* POSIX identifies regular expressions as belonging to two categories — basic and extended.
- \* `grep` supports basic regular expression (BRE) by default and extended regular expression with the `-E` option.
- \* `sed` supports only the BRE.
- \* Regular expressions are interpreted by the command and not by the shell.
- \* Quoting ensures that the shell isn't able to interfere and interpret the metacharacters in its own way.
- \* 

\$ `grep "[aA]g[an][an]wal" emp.lst`



\*=> The \*

- The (asterick \*) refers to the immediately preceding character
- It indicates that the previous character can occur many times, or not at all.

$g^*$

↳ matches the single character  $g$ , on any number of  $g$ s

g gg ggg gggg ...

\* [aA]gg\*[a-zA-Z][a-zA-Z]wal

↳ matches all three patterns.

► \$ grep "[aA]gg\*[a-zA-Z][a-zA-Z]wal" emp.lst



## \* The Dot (.)

[7]

```
$ grep "j.*saxena" emp.lst
```

\* ^ (caret) - For matching at the beginning of a line.

\* \$ - For matching at the end of a line

```
$ grep "^2" emp.lst
```

```
$ grep "7...$" emp.lst
```

\* How can you reverse the search and select only those lines where the emp-ids don't begin with a 2?

Do - it assignment.

Solution:-

[8]

```
$ grep "^[^2]" emp.lst
```

\* UNIX has no command that lists only directories.

\* However, we can use a pipeline to "grep" those lines from the listing that begin with a d:

```
ls -l | grep "^[d]"
```

\* Important:-

— The caret has a triple role to play in regular expression

\* When placed at the beginning of a character class (e.g. `^[a-z]`), it negates every character of the class.

\* When placed outside it, and at the beginning of the expression (e.g. `^2`) the pattern is matched at the beginning of the line.



## Extended Regular Expressions (ERE) AND egrep :-

\* ERE make it possible to match dissimilar patterns with a single expression.

+ → Matches one or more occurrences of the previous character.

? →

↳ Matches zero or one occurrence of the previous character.

```
$ grep -E "[aA]gg?anwal" emp.lst
```

\* Matching Multiple Patterns (|, (and))

> The "|" is a delimiter of multiple patterns. Using it, we can locate both sengupta and dasgupta from the file and without using the -e option twice:

```
$ grep -E 'sengupta|dasgupta' emp.lst.
```

- \* ERE thus handles the problem easily, but offers an even better alternative.
- \* The characters, ( and ), let you group patterns.

```
$ grep -E '(sen|das)' gupta' emp.lst
```

Imp:- If grep doesn't support the -E on your machine, use egrep without the -E option for all examples considered.

[ terminal-passport ]